

ISIT312/ISIT912 Big Data Management

Python MapReduce Application

Dr Guoxin Su and Dr Janusz R. Getta

School of Computing and Information Technology -
University of Wollongong

Java MapReduce Application

Outline

[Building blocks of MapReduce program](#)

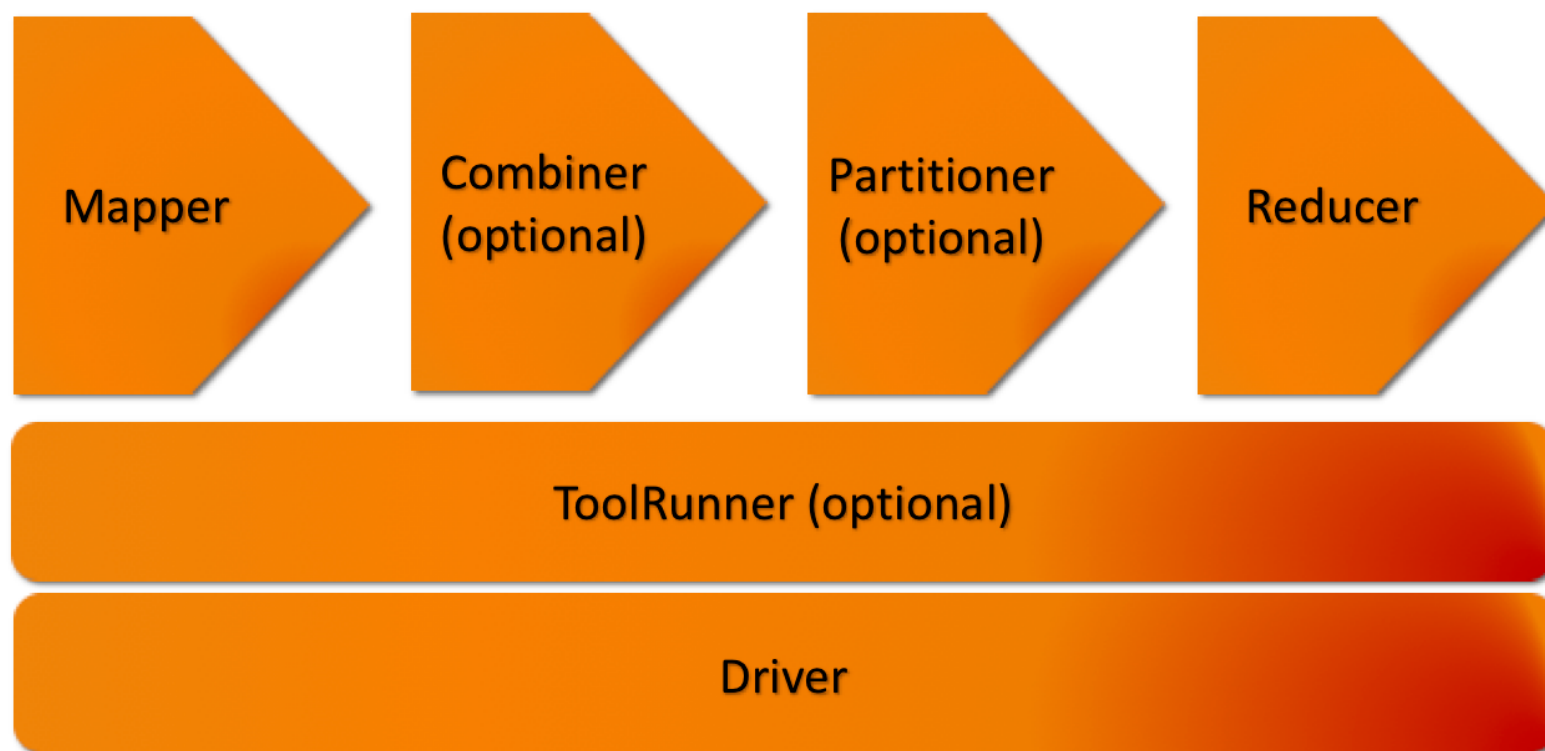
[Hadoop streaming](#)

[Word Count: The "Hello, World" of MapReduce](#)

[Miscellaneous](#)

Building blocks of MapReduce program

Structure of MapReduce Java Application



[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

3/26

Building blocks of MapReduce program

Mapper, **Reducer**, **Combiner**, and **Partitioner** classes correspond to their counterparts in the **MapReduce** model

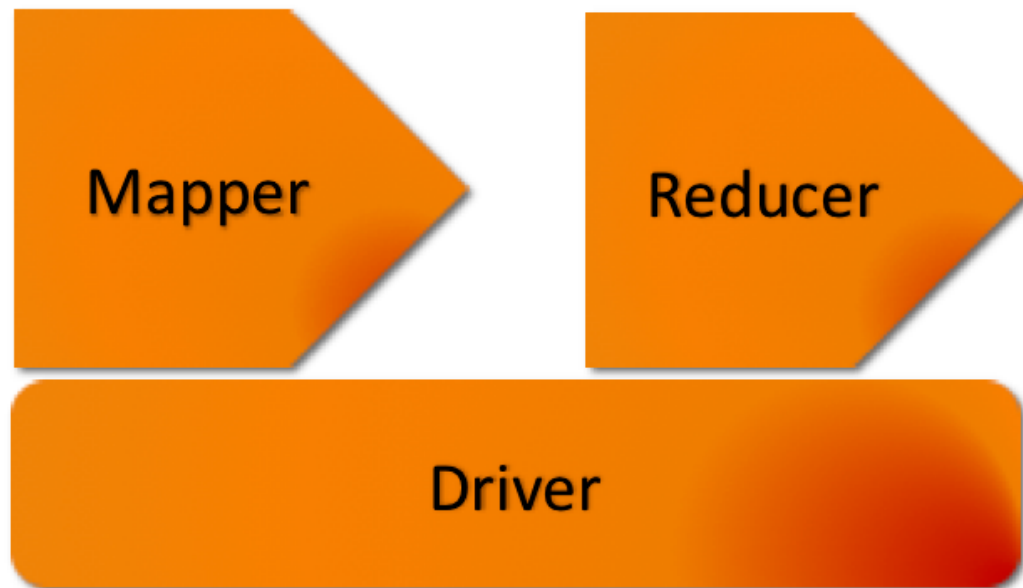
- These classes implement the **MapReduce** logic

The **Driver** or **ToolRunner** in a **MapReduce** program represents the client program

- The main method of a **MapReduce** program is in the **Driver** or **ToolRunner**
- The code of the two is very standard

Building blocks of MapReduce program

An elementary MapReduce program consists only of a Mapper class, a Reducer class and a Driver



As the main method is contained in the Driver, sometimes (but not always) it is convenient to make Mapper and Reducer as inner classes in Driver, which contains routine codes

Building blocks of MapReduce program: Driver

Driver is the program which sets up and starts a **MapReduce** application

Driver code is executed on the client; this code submits the application to the **ResourceManager** along with the application's configuration

Driver can submit the job asynchronously (in a non-blocking fashion) or synchronously (waiting for the application to complete before performing another action)– the second way will be used in our examples

Driver can also configure and submit more than one application; for instance, running a workflow consisting of multiple **MapReduce** applications

Building blocks of MapReduce program: Mapper

Mapper Java class contains a `map( )` method

Its object instance iterates through the input to execute a `map( )` method, using the **InputFormat** and its associated **RecordReader**

The number of **HDFS** blocks for the file determines the number of input splits, which, in turn, determines the number of **Mapper** objects (or **Map** tasks) in a **MapReduce** application (also see the previous lecture)

Mappers do most of the heavy lifting in data processing in **MapReduce**, as they read the entire input file for the application

Mappers can also include setup and cleanup code to run in any given object lifespan

Setup is called before the `map( )` method; cleanup is called after it

Building blocks of MapReduce program: Reducer

Reducer runs against a partition and each key and its associated values are passed to a `reduce ( )` method inside **Reducer class**

Reduce's InputFormat matches **Mapper's** OutputFormat

While **Mapper** usually do the data preparation, for example, filtering and extracting), **Reducer** usually contains the main application logic

- For example, summation, counting, and averaging operations are implemented in **Reducers**

The runtime of **Reducer** instances is usually faster (and much faster in some cases) than the runtime of **Mapper** instances

Java MapReduce Application

Outline

[Building blocks of MapReduce program](#)

[Hadoop streaming](#)

[Word Count: The "Hello, World" of MapReduce](#)

[Miscellaneous](#)

Hadoop streaming

Hadoop streaming is an utility that allows to create and process Map/Reduce jobs with any executable or script as the mapper and/or the reducer

For example counting lines and words and characters in a file located in **Input** folder in **HDFS**

```
mapred streaming -input Input -output Output -mapper /bin/cat -reducer /usr/bin/wc
```

Sample streaming

mapred is an utility located in **\$HADOOP_HOME/bin**

A parameter **-input** determines input folder or file name in **HDFS**

A parameter **-output** determines output folder in **HDFS**

A parameter **-mapper** determines executable or Java Class name

A parameter **-reducer** determines executable or Java Class name

cat is a program that concatenates and prints the files to standard output

wc is a program that counts lines, words, and characters in each input file

[In HTML view press 'p' to see the lecture notes](#)
[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

10/26

Hadoop streaming

In the example below

```
mapred streaming -input /user/words.txt -output /user/output -mapper /bin/cat -reducer /usr/bin/wc
```

the **mapper** and the **reducer** are executables that read the input from standard input (line by line) and emit the output to standard output

mapred utility creates a Map/Reduce job, submits the job to a cluster, and monitors processing of the job until it completed

Let a file **words.txt** in **/user** in **HDFS** contains

```
word1 word2  
word3
```

Then the outcomes from **mapred** in **/user/output/part-00000** are the following

```
2    3    20
```

Hadoop streaming

Processing of mapper is performed in the following way:

- A mapper starts processing the executable (in our example `/bin/cat`) as a separate process when the mapper is initialized
- The mapper converts its inputs into lines and feeds the lines to the standard input of the process
- Simultaneously, the mapper collects the lines from the standard output of the process and converts each line into a `key/value` pair
- The `key/value` pairs are collected as the output of the mapper
- By default, the prefix of a line up to the first tab character is the `key` and the rest of the line (excluding the tab character) is the `value`

Hadoop streaming

Processing of reducer is performed in the following way:

- A reducer task starts its executable (in our example `/user/bin/wc` as a separate process
- The reducer task converts its input key/values pairs into lines and submits the lines to the standard input of the process
- Simultaneously, the reducer collects the lines from the standard output of the process and converts each line into a `key/value` pair
- The `key/value` is collected as the output of the reducer
- By default, the prefix of a line up to the first tab character is the key and the rest of the line (excluding the tab character) is the value

Hadoop streaming

Hadoop streaming supports **streaming command options** as well as **generic command options**

Options

```
mapred streaming [genericOptions] [streamingOptions]
```

Generic command options include

Generic command options

<code>-conf configuration_file</code>	Specify an application configuration file
<code>-D property=value</code>	Use value for given property
<code>-fs host:port or local</code>	Specify a namenode
<code>-files Optional</code>	Specify comma-separated files to be copied to the Map/Reduce cluster
<code>-libjars</code>	Specify comma-separated jar files to include in the classpath
<code>-archives</code>	Specify comma-separated archives to be unarchived on the compute machines

In HTML view press 'p' to see the lecture notes

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

14/26

Hadoop streaming

Sample applications of **generic command options**

Specifying map-only jobs

```
-D mapreduce.job.reduces=0
```

Map-only

Specifying number of reducers

```
-D mapreduce.job.reduces=2
```

Number of reducers

Specifying how lines are Split into **key/value** Pairs

```
-D stream.map.output.field.separator=.  
-D stream.num.map.output.key.fields=4
```

key/value pairs

[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

15/26

Hadoop streaming

Streaming command options include

Generic command options

-input directoryname or filename	Input location for mapper
-output directoryname Required	Output location for reducer
-mapper executable or JavaClassName	Mapper executable, default IdentityMapper
-reducer executable or JavaClassName	Reducer executable, default IdentityMapper
-file filename	Make the mapper, reducer, or combiner executable available locally
-inputformat JavaClassName	Class you supply should return key/value pairs of Text class
-outputformat JavaClassName	Class you supply should take key/value pairs of Text class
-partitioner JavaClassName	Class that determines which reduce a key is sent to
-combiner streamingCommand or JavaClassName	Combiner executable for map output
-cmdenv name=value	Pass environment variable to streaming commands
-inputreader	For backwards-compatibility: specifies a record reader class
-verbose	Verbose output
-lazyOutput	Create output lazily
-numReduceTasks	Specify the number of reducers
-mapdebug	Script to call when map task fails
-reduceddebug	Script to call when reduce task fails

Hadoop streaming

Sample applications of **streaming command options**

Specifying Java class as the mapper and/or the reducer

Java class as mapper

```
mapred streaming -input Input -output Output \  
                  -inputformat org.apache.hadoop.mapred.KeyValueTextInputFormat \  
                  -mapper org.apache.hadoop.mapred.lib.IdentityMapper \  
                  -reducer /usr/bin/wc
```

Packaging files with job submissions

Packaging files

```
mapred streaming -input Input -output Output \  
                  -mapper myPythonScript.py \  
                  -reducer /usr/bin/wc \  
                  -file myPythonScript.py
```

Specifying other plugins for jobs

Other plugins

```
-inputformat JavaClassName  
-outputformat JavaClassName  
-partitioner JavaClassName  
-combiner streamingCommand or JavaClassName
```

[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

17/26

Java MapReduce Application

Outline

[Building blocks of MapReduce program](#)

[Hadoop streaming](#)

[Word Count: The "Hello, World" of MapReduce](#)

[Miscellaneous](#)

Word Count: The "Hello, World" of MapReduce

Implementation of word count program in Python

Implementation of mapper `mapper.py`

```
#!/usr/bin/env python3

import sys

for line in sys.stdin:
    line = line.strip()
    words = line.split()
    for word in words:
        print('%s\t%s' % (word, 1))
```

Mapper

Word Count: The "Hello, World" of MapReduce

Implementation of reducer `reducer.py`

```
#!/usr/bin/env python3

import sys

current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word, count = line.split('\t', 1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
            current_count = count
            current_word = word
if current_word == word:
    print('%s\t%s' % (current_word, current_count))
```

Reducer

[In HTML view](#) press 'p' to see the lecture notes
[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

20/26

Word Count: The "Hello, World" of MapReduce

Implementation of word count program in modern Python

Implementation of mapper `mapper.py`

```
#!/usr/bin/env python3

import sys

def read_input(file):
    for line in file:
        yield line.split()

def main(separator='\t'):
    data = read_input(sys.stdin)
    for words in data:
        for word in words:
            print '%s%s%d' % (word, separator, 1)

if __name__ == "__main__":
    main()
```

Mapper

[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

21/26

Word Count: The "Hello, World" of MapReduce

Implementation of reducer `reducer.py`

```
#!/usr/bin/env python3

from itertools import groupby
from operator import itemgetter
import sys

def read_mapper_output(file, separator='\t'):
    for line in file:
        yield line.rstrip().split(separator, 1)

def main(separator='\t'):
    data = read_mapper_output(sys.stdin, separator=separator)
    for current_word, group in groupby(data, itemgetter(0)):
        try:
            total_count = sum(int(count) for current_word, count in group)
            print "%s%s%d" % (current_word, separator, total_count)
        except ValueError:
            pass

if __name__ == "__main__":
    main()
```

Reducer

Java MapReduce Application

Outline

[Building blocks of MapReduce program](#)

[Hadoop streaming](#)

[Word Count: The "Hello, World" of MapReduce](#)

[Miscellaneous](#)

Word Count: The "Hello, World" of MapReduce

Sample processing of MapReduce job

```
mapred streaming -input /user/words.txt -output /user/output \  
-mapper ./mapper.py -reducer ./reducer.py
```

MapReduce

or

```
$HADOOP_HOME/bin/hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \  
-input /user/numbers.txt -output /user/outputm \  
-mapper ./filter.py -reducer ./reducer.py
```

MapReduce

Miscellaneous

Compound **MapReduce** jobs

How to turn a (possibly complex) data processing problem into the **MapReduce model** ?

As a rule of thumb, for complex data processing problems, think about adding more jobs, rather than adding the complexity to jobs

For example, **to find the mean of maximum daily temperature in each month in different weather stations**, consider these two jobs:

- Compute the maximum daily temperature for every station-day key
- Compute the mean of the maximum daily temperature for every station-day-month key

A library **ChainMapper** allows to develop chained MapReduce Jobs

References

White T., Hadoop The Definitive Guide: Storage and Analysis at Internet Scale, O'Reilly, 2015

[Hadoop Streaming](#)

[Hadoop with Python](#)

[Writing An Hadoop MapReduce Program In Python](#)